

🐍 Snake RL: Multi-Algorithm Comparison

EE 473 — Reinforcement Learning | Northwestern University

Authors: Willie Liao, Wei-In Lai

Youtube link : https://www.youtube.com/watch?v=vaQDU36M_Q

1. Overview & Course Fit

This project aims to solve the classic game of Snake using Reinforcement Learning (RL). We implement and compare multiple distinct RL approaches to optimize the agent's behavior in a grid-based environment. This project perfectly aligns with the course requirements:

- **Solving an RL problem with a moderately large dataset:** We generate an expert dataset via simulation and employ Offline Imitation Learning.
- **Comparing algorithms:** We leverage tabular methods (Q-Learning, SARSA) as baselines and compare them against function approximation (Neural Networks) via Deep Q-Network (DQN) and PPO.

Problem & Objectives

The primary goal is to train an agent that maximizes the game score while avoiding collisions with walls or its own body. Our objectives include:

1. Developing a simulation environment to track the state space (snake head, food, and obstacles).
2. Implementing and comparing Tabular baselines against Deep RL.
3. Analyzing how each algorithm performs in terms of training time, stability, and decision-making accuracy.
4. Comparing a simple sparse reward system with a more complex reward-shaping method.
5. Performing a hyperparameter sweep over the learning rate (α) and discount factor (γ).

```
# Install required packages (Colab-ready)
!pip install gymnasium stable-baselines3 numpy matplotlib pandas --quiet
```

188.0/188.0 kB 5.9 MB/s eta 0:00:00

```
import time
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import gymnasium as gym
from gymnasium import spaces
import random
import pickle
import warnings
warnings.filterwarnings("ignore")

matplotlib.rcParams.update({
    "font.size": 12,
    "axes.grid": True,
    "grid.alpha": 0.3,
})

print("✅ All imports successful")
```

✅ All imports successful

2. Environment Design

We implement a **custom Gymnasium environment** for Snake with a compact state representation:

State Space (11-dimensional boolean vector)

Index	Feature	Description
0-2	Danger	Collision ahead (straight / right / left)
3-6	Direction	Current heading (left / right / up / down)
7-10	Food	Relative food position (left / right / up / down)

- **Total states:** $2^{11} = 2048$ — perfectly suited for tabular methods
- **Action space:** 3 discrete actions — *go straight, turn right, turn left*

- **Reward:** +10 (food), -10 (death), 0 (otherwise)
- **Reward Shaping** (optional): +0.1 / -0.1 based on Manhattan distance change to food

```

class SnakeEnv(gym.Env):
    # Custom Snake environment for RL training (headless)
    metadata = {"render_modes": [], "render_fps": 15}

    def __init__(self, render_mode=None, grid_size=20, block_size=20, reward_shaping=False):
        super().__init__()
        self.grid_size = grid_size
        self.block_size = block_size
        self.reward_shaping = reward_shaping
        self.action_space = spaces.Discrete(3)
        self.observation_space = spaces.Box(low=0, high=1, shape=(11,), dtype=np.int8)
        self.render_mode = render_mode
        self.reset()

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.head = [self.grid_size // 2, self.grid_size // 2]
        self.snake = [self.head.copy(),
                      [self.head[0] - 1, self.head[1]],
                      [self.head[0] - 2, self.head[1]]]
        self.direction = [1, 0]
        self.food = self._place_food()
        self.score = 0
        self.frame_iteration = 0
        return self._get_state(), {}

    def _place_food(self):
        while True:
            food = [random.randint(0, self.grid_size - 1),
                    random.randint(0, self.grid_size - 1)]
            if food not in self.snake:
                return food

    def step(self, action):
        self.frame_iteration += 1
        clock_wise = [[1, 0], [0, 1], [-1, 0], [0, -1]]
        idx = clock_wise.index(self.direction)
        if action == 0:
            new_dir = clock_wise[idx]
        elif action == 1:
            new_dir = clock_wise[(idx + 1) % 4]
        else:
            new_dir = clock_wise[(idx - 1) % 4]
        self.direction = new_dir
        self.head = [self.head[0] + self.direction[0],
                    self.head[1] + self.direction[1]]
        self.snake.insert(0, self.head.copy())

        reward, terminated = 0, False
        if self._is_collision() or self.frame_iteration > 100 * len(self.snake):
            terminated = True
            reward = -10
            return self._get_state(), reward, terminated, False, {}
        if self.head == self.food:
            self.score += 1
            reward = 10
            self.food = self._place_food()
        else:
            self.snake.pop()
            if self.reward_shaping:
                old_d = abs(self.snake[1][0] - self.food[0]) + abs(self.snake[1][1] - self.food[1])
                new_d = abs(self.head[0] - self.food[0]) + abs(self.head[1] - self.food[1])
                reward = 0.1 if new_d < old_d else -0.1
            return self._get_state(), reward, terminated, False, {}

    def _is_collision(self, pt=None):
        if pt is None:
            pt = self.head
        if pt[0] < 0 or pt[0] >= self.grid_size or pt[1] < 0 or pt[1] >= self.grid_size:
            return True
        return pt in self.snake[1:]

    def _get_state(self):
        hx, hy = self.head
        cw = [[1, 0], [0, 1], [-1, 0], [0, -1]]
        idx = cw.index(self.direction)
        ds, dr, dl = cw[idx], cw[(idx+1)%4], cw[(idx-1)%4]
        state = [

```

```

        int(self._is_collision([hx+ds[0], hy+ds[1]])),
        int(self._is_collision([hx+dr[0], hy+dr[1]])),
        int(self._is_collision([hx+dl[0], hy+dl[1]])),
        int(self.direction == [-1, 0]),
        int(self.direction == [1, 0]),
        int(self.direction == [0, -1]),
        int(self.direction == [0, 1]),
        int(self.food[0] < hx), int(self.food[0] > hx),
        int(self.food[1] < hy), int(self.food[1] > hy),
    ]
    return np.array(state, dtype=np.int8)

def close(self):
    pass

# Quick sanity check
env = SnakeEnv()
obs, _ = env.reset()
print(f"Observation shape: {obs.shape} | Sample: {obs}")
print(f"Action space: {env.action_space}")
print("✅ SnakeEnv works!")

```

```

Observation shape: (11,) | Sample: [0 0 0 0 1 0 0 0 1 0 1]
Action space: Discrete(3)
✅ SnakeEnv works!

```

```

def state_to_int(state_vec):
    # Convert 11-dim boolean vector to a single integer index (0-2047)
    val = 0
    for i, bit in enumerate(state_vec):
        val += int(bit) * (2 ** i)
    return val

# Smooth helper for plotting
def smooth(values, window=50):
    if len(values) < window:
        return values
    return np.convolve(values, np.ones(window) / window, mode="valid")

```

3. Algorithm Implementations

3.1 Tabular Q-Learning (Off-Policy)

Q-Learning updates using the **maximum** next-state Q-value, regardless of the action actually taken. This makes it **off-policy** — tending toward *optimistic*, risk-seeking behavior.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

```

class TabularQ:
    def __init__(self, state_size, action_size, lr=0.1, gamma=0.99,
                 epsilon=1.0, epsilon_min=0.01, epsilon_decay=0.995):
        self.q_table = np.zeros((state_size, action_size))
        self.lr = lr
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.action_size = action_size

    def choose_action(self, state):
        if np.random.rand() <= self.epsilon:
            return np.random.choice(self.action_size)
        return np.argmax(self.q_table[state])

    def learn(self, state, action, reward, next_state, done):
        best_next = np.argmax(self.q_table[next_state])
        td_target = reward + self.gamma * self.q_table[next_state][best_next] * (not done)
        self.q_table[state][action] += self.lr * (td_target - self.q_table[state][action])
        if done and self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

```

3.2 SARSA (On-Policy)

SARSA updates using the **actual** next action (including exploration), making it **on-policy**. This results in more *conservative*, risk-averse behavior compared to Q-Learning.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

```
class TabularSARSA:
    def __init__(self, state_size, action_size, lr=0.1, gamma=0.99,
                 epsilon=1.0, epsilon_min=0.01, epsilon_decay=0.995):
        self.q_table = np.zeros((state_size, action_size))
        self.lr = lr
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.action_size = action_size

    def choose_action(self, state):
        if np.random.rand() <= self.epsilon:
            return np.random.choice(self.action_size)
        return int(np.argmax(self.q_table[state]))

    def learn(self, state, action, reward, next_state, next_action, done):
        td_target = reward + self.gamma * self.q_table[next_state][next_action] * (not done)
        self.q_table[state][action] += self.lr * (td_target - self.q_table[state][action])
        if done and self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

3.3 DQN (Deep Q-Network)

DQN approximates the Q-function with a neural network. Key techniques:

- **Experience Replay**: stores transitions in a buffer and samples mini-batches to break correlation
- **Target Network**: a slowly-updated copy of the Q-network to stabilize training

3.4 PPO (Proximal Policy Optimization)

PPO directly optimizes the policy using the **clipped surrogate objective**:

$$L^{CLIP}(\theta) = \mathbb{E} \left[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.

Both DQN and PPO are implemented using **Stable-Baselines3**.

4. Training All Algorithms

We train each algorithm and record:

- **Per-episode scores** (number of food items eaten)
- **Wall-clock training time**

```
EPISODES_TAB = 2000
STATE_SIZE = 2 ** 11
ACTION_SIZE = 3

def train_tabular(AgentClass, name, episodes, reward_shaping=False, is_sarsa=False):
    # Train a tabular agent and return (scores, elapsed_seconds)
    env = SnakeEnv(reward_shaping=reward_shaping)
    agent = AgentClass(STATE_SIZE, ACTION_SIZE)
    scores = []
    t0 = time.time()

    for ep in range(episodes):
        state_vec, _ = env.reset()
        state = state_to_int(state_vec)
        done, score = False, 0

        if is_sarsa:
            action = agent.choose_action(state)
            while not done:
                ns_vec, reward, done, _, _ = env.step(action)
                ns = state_to_int(ns_vec)
                na = agent.choose_action(ns)
                agent.learn(state, action, reward, ns, na, done)
                state, action = ns, na
```

```

        if reward == 10:
            score += 1
    else:
        while not done:
            action = agent.choose_action(state)
            ns_vec, reward, done, _, _ = env.step(action)
            ns = state_to_int(ns_vec)
            agent.learn(state, action, reward, ns, done)
            state = ns
            if reward == 10:
                score += 1
        scores.append(score)

    if (ep + 1) % 500 == 0:
        print(f" [{name}] Episode {ep+1}/{episodes} - "
              f"avg(last 100): {np.mean(scores[-100:]).1f}")

    elapsed = time.time() - t0
    print(f" [{name}] Done in {elapsed:.1f}s - "
          f"final avg: {np.mean(scores[-100:]).2f}\n")
    return scores, elapsed, agent

all_curves = {}
train_times = {}
trained_agents = {}

# --- Q-Learning ---
print("► Training Tabular Q-Learning ...")
all_curves["Q-Learning"], train_times["Q-Learning"], trained_agents["Q-Learning"] = \
    train_tabular(TabularQ, "Q-Learning", EPISODES_TAB)

# --- SARSA ---
print("► Training SARSA ...")
all_curves["SARSA"], train_times["SARSA"], trained_agents["SARSA"] = \
    train_tabular(TabularSARSA, "SARSA", EPISODES_TAB, is_sarsa=True)

# --- Q-Learning + Reward Shaping ---
print("► Training Q-Learning + Reward Shaping ...")
all_curves["Q-Learning + RS"], train_times["Q-Learning + RS"], trained_agents["Q-Learning + RS"] = \
    train_tabular(TabularQ, "Q-Learning+RS", EPISODES_TAB, reward_shaping=True)

```

```

► Training Tabular Q-Learning ...
[Q-Learning] Episode 500/2000 - avg(last 100): 6.2
[Q-Learning] Episode 1000/2000 - avg(last 100): 19.5
[Q-Learning] Episode 1500/2000 - avg(last 100): 23.3
[Q-Learning] Episode 2000/2000 - avg(last 100): 21.4
[Q-Learning] Done in 10.1s - final avg: 21.38

► Training SARSA ...
[SARSA] Episode 500/2000 - avg(last 100): 7.2
[SARSA] Episode 1000/2000 - avg(last 100): 22.5
[SARSA] Episode 1500/2000 - avg(last 100): 22.6
[SARSA] Episode 2000/2000 - avg(last 100): 17.0
[SARSA] Done in 8.1s - final avg: 17.00

► Training Q-Learning + Reward Shaping ...
[Q-Learning+RS] Episode 500/2000 - avg(last 100): 6.7
[Q-Learning+RS] Episode 1000/2000 - avg(last 100): 21.9
[Q-Learning+RS] Episode 1500/2000 - avg(last 100): 22.7
[Q-Learning+RS] Episode 2000/2000 - avg(last 100): 24.7
[Q-Learning+RS] Done in 9.5s - final avg: 24.71

```

```

import torch
from stable_baselines3 import DQN
from stable_baselines3.common.monitor import Monitor

TIMESTEPS_DL = 200_000

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"► Training DQN on {device} for {TIMESTEPS_DL} timesteps ...")

env_dqn = Monitor(SnakeEnv())
t0 = time.time()
model_dqn = DQN("MlpPolicy", env_dqn, verbose=0, device=device,
               learning_rate=1e-4, gamma=0.99, buffer_size=100_000,
               exploration_fraction=0.1, exploration_final_eps=0.05)
model_dqn.learn(total_timesteps=TIMESTEPS_DL)
train_times["DQN"] = time.time() - t0

# Extract per-episode scores from Monitor
ep_rewards_dqn = env_dqn.get_episode_rewards()
all_curves["DQN"] = [max(0, int((r + 10) / 10)) for r in ep_rewards_dqn]

```

```
print(f" [DQN] Done in {train_times['DQN']:.1f}s - "
      f"{len(all_curves['DQN'])} episodes - "
      f"final avg: {np.mean(all_curves['DQN'][-100:]):.2f}\n")
```

Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality. Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software. See the migration guide at https://gymnasium.farama.org/introduction/migration_guide/ for additional information.

► Training DQN on cuda for 200000 timesteps ...

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
return datetime.utcnow().replace(tzinfo=utc)
[DQN] Done in 242.3s - 1137 episodes - final avg: 13.89
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
return datetime.utcnow().replace(tzinfo=utc)
```

```
from stable_baselines3 import PPO
```

```
print(f"► Training PPO on {device} for {Timesteps_DL} timesteps ...")
```

```
env_ppo = Monitor(SnakeEnv())
t0 = time.time()
model_ppo = PPO("MlpPolicy", env_ppo, verbose=0, device=device,
                learning_rate=3e-4, n_steps=2048, batch_size=64,
                n_epochs=10, clip_range=0.2, ent_coef=0.01)
model_ppo.learn(total_timesteps=Timesteps_DL)
train_times['PPO'] = time.time() - t0
```

```
ep_rewards_ppo = env_ppo.get_episode_rewards()
all_curves['PPO'] = [max(0, int((r + 10) / 10)) for r in ep_rewards_ppo]
print(f" [PPO] Done in {train_times['PPO']:.1f}s - "
      f"{len(all_curves['PPO'])} episodes - "
      f"final avg: {np.mean(all_curves['PPO'][-100:]):.2f}\n")
```

► Training PPO on cuda for 200000 timesteps ...

```
[PPO] Done in 433.2s - 679 episodes - final avg: 24.45
```

5. Hyperparameter Sensitivity Analysis

We sweep over key Q-Learning hyperparameters to study their effect:

Parameter	Values
Learning rate α	0.05, 0.1, 0.2
Discount factor γ	0.9, 0.95, 0.99
ϵ -decay	0.99, 0.995, 0.999

Each configuration runs for **1000 episodes**; we report the average score over the last 100 episodes.

```
import itertools
import pandas as pd

SWEEP_EPISODES = 1000
lrs = [0.05, 0.1, 0.2]
gammas = [0.9, 0.95, 0.99]
eps_decays = [0.99, 0.995, 0.999]

results = []
total = len(lrs) * len(gammas)
print(f"--- 1. Running 2D Grid Search ( $\alpha$  vs  $\gamma$ ) ---")
print(f"Configurations: {total} × {SWEEP_EPISODES} episodes\n")

for i, (lr, gamma) in enumerate(itertools.product(lrs, gammas)):
    env_sw = SnakeEnv()
    agent = TabularQ(STATE_SIZE, ACTION_SIZE, lr=lr, gamma=gamma, epsilon_decay=0.995)
    scores = []
    for _ in range(SWEEP_EPISODES):
        sv, _ = env_sw.reset()
        s = state_to_int(sv)
        done, sc = False, 0
        while not done:
            a = agent.choose_action(s)
            ns, r, done, _, _ = env_sw.step(a)
            ns = state_to_int(ns)
            agent.learn(s, a, r, ns, done)
            s = ns
            if r == 10:
                sc += 1
```

```

        scores.append(sc)
    avg = np.mean(scores[-100:])
    results.append({"lr": lr, "gamma": gamma, "avg_score": round(avg, 2)})
    print(f" [{i+1}/{total}]  $\alpha$ ={lr},  $\gamma$ ={gamma}  $\rightarrow$  avg={avg:.2f}")

df = pd.DataFrame(results)
best = df.loc[df["avg_score"].idxmax()]
print(f"\n★ Best:  $\alpha$ ={best['lr']},  $\gamma$ ={best['gamma']}  $\rightarrow$  avg score={best['avg_score']}")

# ---- Heatmap ----
pivot = df.pivot(index="lr", columns="gamma", values="avg_score")
fig, ax = plt.subplots(figsize=(7, 4))
im = ax.imshow(pivot.values, cmap="YlOrRd", aspect="auto")
ax.set_xticks(range(len(pivot.columns)))
ax.set_xticklabels(pivot.columns)
ax.set_yticks(range(len(pivot.index)))
ax.set_yticklabels(pivot.index)
ax.set_xlabel("Discount Factor ( $\gamma$ )")
ax.set_ylabel("Learning Rate ( $\alpha$ )")
ax.set_title("Hyperparameter Sensitivity - Avg Score (last 100 ep)")
for r in range(len(pivot.index)):
    for c in range(len(pivot.columns)):
        ax.text(c, r, f"{pivot.values[r, c]:.1f}",
                ha="center", va="center", fontsize=12, fontweight="bold")
fig.colorbar(im, ax=ax)
fig.tight_layout()
plt.show()

print("\n--- 2. Running 1D Search ( $\epsilon$ -decay) ----")
eps_results = []
best_alpha, best_gamma = best['lr'], best['gamma']
for ed in eps_decays:
    env_sw = SnakeEnv()
    agent = TabularQ(STATE_SIZE, ACTION_SIZE, lr=best_alpha, gamma=best_gamma, epsilon_decay=ed)
    scores = []
    for _ in range(SWEEP_EPISODES):
        sv, _ = env_sw.reset()
        s = state_to_int(sv)
        done, sc = False, 0
        while not done:
            a = agent.choose_action(s)
            nsv, r, done, _, _ = env_sw.step(a)
            ns = state_to_int(nsv)
            agent.learn(s, a, r, ns, done)
            s = ns
            if r == 10: sc += 1
        scores.append(sc)
    avg = np.mean(scores[-100:])
    eps_results.append((ed, avg))
    print(f"  $\epsilon$ -decay={ed}  $\rightarrow$  avg={avg:.2f}")

best_ed = max(eps_results, key=lambda x: x[1])
print(f"\n★ Best  $\epsilon$ -decay: {best_ed[0]} (avg {best_ed[1]:.2f})")

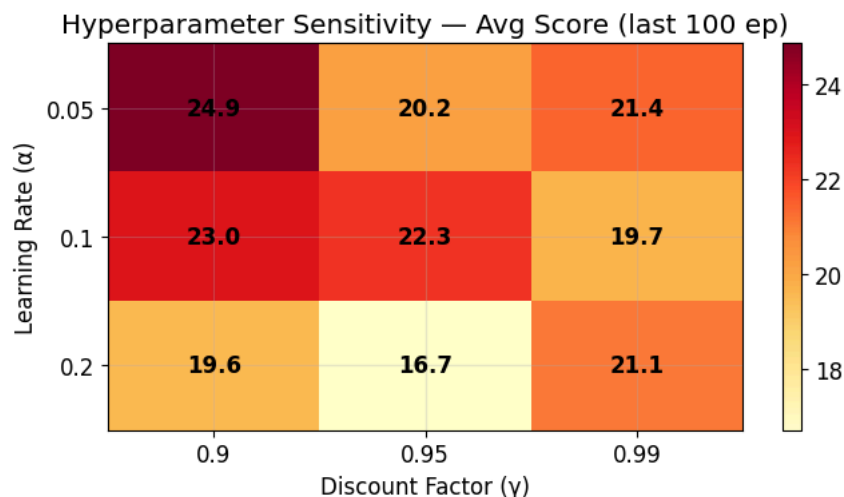
```

--- 1. Running 2D Grid Search (α vs γ) ---
 Configurations: 9×1000 episodes

```
[1/9]  $\alpha=0.05$ ,  $\gamma=0.9 \rightarrow \text{avg}=24.86$ 
[2/9]  $\alpha=0.05$ ,  $\gamma=0.95 \rightarrow \text{avg}=20.18$ 
[3/9]  $\alpha=0.05$ ,  $\gamma=0.99 \rightarrow \text{avg}=21.43$ 
[4/9]  $\alpha=0.1$ ,  $\gamma=0.9 \rightarrow \text{avg}=23.01$ 
[5/9]  $\alpha=0.1$ ,  $\gamma=0.95 \rightarrow \text{avg}=22.26$ 
[6/9]  $\alpha=0.1$ ,  $\gamma=0.99 \rightarrow \text{avg}=19.69$ 
[7/9]  $\alpha=0.2$ ,  $\gamma=0.9 \rightarrow \text{avg}=19.55$ 
[8/9]  $\alpha=0.2$ ,  $\gamma=0.95 \rightarrow \text{avg}=16.71$ 
[9/9]  $\alpha=0.2$ ,  $\gamma=0.99 \rightarrow \text{avg}=21.06$ 
```

★ Best: $\alpha=0.05$, $\gamma=0.9 \rightarrow \text{avg score}=24.86$

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use datetime.datetime.now(datetime.timezone.utc) instead.



--- 2. Running 1D Search (ϵ -decay) ---

```
 $\epsilon$ -decay=0.99  $\rightarrow \text{avg}=21.00$ 
 $\epsilon$ -decay=0.995  $\rightarrow \text{avg}=20.53$ 
 $\epsilon$ -decay=0.999  $\rightarrow \text{avg}=1.98$ 
```

★ Best ϵ -decay: 0.99 (avg 21.00)

6. Project with a Dataset: Imitation Learning (Behavior Cloning)

To satisfy the "Project with a Dataset" requirement, we introduce **Offline Imitation Learning**. Instead of learning by interacting with the environment (Online RL), the agent learns purely from a static dataset of expert demonstrations.

6.1 Generating the Expert Dataset

We use our best-performing agent (Tabular Q-Learning) to play 1,000 games and record every (State, Action) transition. This creates a moderately large dataset with hundreds of thousands of samples.

```
print("► Generating Expert Dataset using Trained Q-Learning Agent...")
expert_agent = trained_agents["Q-Learning + RS"]
expert_agent.epsilon = 0.0 # Fully greedy (expert behavior)

dataset = {"states": [], "actions": []}
env_data = SnakeEnv(reward_shaping=False)

for _ in range(1000):
    sv, _ = env_data.reset()
    s = state_to_int(sv)
    done = False
    while not done:
        a = expert_agent.choose_action(s)
        dataset["states"].append(s)
        dataset["actions"].append(a)

        ns, r, done, _, _ = env_data.step(a)
        s = state_to_int(ns)

    dataset_size = len(dataset["states"])
    print(f"✅ Generated dataset with {dataset_size} state-action transitions.")
```

► Generating Expert Dataset using Trained Q-Learning Agent...

✅ Generated dataset with 455532 state-action transitions.

6.2 Tabular Behavior Cloning (BC)

We implement a simple **Behavior Cloning** algorithm which counts the frequency of actions taken by the expert in each state.

Algorithm:

$$\pi_{BC}(a|s) = \frac{N(s, a)}{N(s)}$$

where $N(s, a)$ is the number of times the expert took action a in state s .

```
class TabularBC:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.action_counts = np.zeros((state_size, action_size))
        self.policy = np.zeros((state_size, action_size))
        self.is_trained = False

    def train_on_dataset(self, dataset):
        t0 = time.time()
        for s, a in zip(dataset["states"], dataset["actions"]):
            self.action_counts[s, a] += 1

        # Normalize to get probabilities
        for s in range(self.state_size):
            total = np.sum(self.action_counts[s])
            if total > 0:
                self.policy[s] = self.action_counts[s] / total
            else:
                self.policy[s] = np.ones(self.action_size) / self.action_size

        self.is_trained = True
        elapsed = time.time() - t0
        print(f"✅ Behavior Cloning trained in {elapsed:.4f} seconds.")
        return elapsed

    def choose_action(self, state):
        return np.random.choice(self.action_size, p=self.policy[state])

# Train BC Agent
bc_agent = TabularBC(STATE_SIZE, ACTION_SIZE)
bc_train_time = bc_agent.train_on_dataset(dataset)

# Evaluate BC Agent (Online)
print("\n► Evaluating Behavior Cloning Agent for 2000 episodes...")
bc_scores = []
env_eval = SnakeEnv()
for _ in range(EPIISODES_TAB):
    sv, _ = env_eval.reset()
    s = state_to_int(sv)
    done, sc = False, 0
    while not done:
        a = bc_agent.choose_action(s)
        nsv, r, done, _, _ = env_eval.step(a)
        s = state_to_int(nsv)
        if r == 10: sc += 1
    bc_scores.append(sc)

bc_avg = np.mean(bc_scores[-100:])
print(f" [Behavior Cloning] Final avg score (last 100): {bc_avg:.2f}\n")

all_curves["Behavior Cloning"] = bc_scores
train_times["Behavior Cloning"] = bc_train_time
```

✅ Behavior Cloning trained in 0.1844 seconds.

► Evaluating Behavior Cloning Agent for 2000 episodes...
[Behavior Cloning] Final avg score (last 100): 30.08

6.3 Impact of Dataset Size

How does the amount of expert data affect the Behavior Cloning agent? To answer this, we train the BC algorithm on varying fractions of the full dataset (from 10% to 100%) and compare their final performance.

```
fractions = [0.1, 0.2, 0.4, 0.6, 0.8, 1.0]
bc_size_results = []
```

```

print("▶ Running Dataset Size Ablation for Behavior Cloning...")
for frac in fractions:
    slice_idx = int(dataset_size * frac)
    sub_dataset = {
        "states": dataset["states"][:slice_idx],
        "actions": dataset["actions"][:slice_idx]
    }

    # Train
    temp_bc = TabularBC(STATE_SIZE, ACTION_SIZE)
    temp_bc.train_on_dataset(sub_dataset)

    # Evaluate
    temp_scores = []
    env_eval = SnakeEnv()
    for _ in range(500): # Evaluate for 500 episodes
        sv, _ = env_eval.reset()
        s = state_to_int(sv)
        done, sc = False, 0
        while not done:
            a = temp_bc.choose_action(s)
            ns, r, done, _, _ = env_eval.step(a)
            s = state_to_int(ns)
            if r == 10: sc += 1
        temp_scores.append(sc)

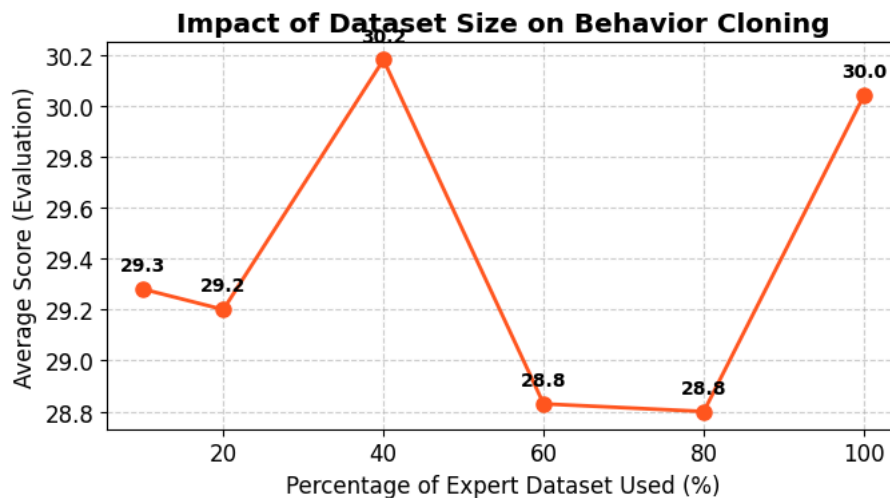
    avg_score = np.mean(temp_scores[-100:])
    bc_size_results.append(avg_score)
    print(f" Fraction: {frac*100:3.0f}% ({slice_idx} samples) -> Avg Score: {avg_score:.2f}")

# Plot the ablation
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot([f * 100 for f in fractions], bc_size_results, marker='o', linestyle='-', color='#FF5722', linewidth=2,
        ax.set_title("Impact of Dataset Size on Behavior Cloning", fontweight="bold")
ax.set_xlabel("Percentage of Expert Dataset Used (%)")
ax.set_ylabel("Average Score (Evaluation)")
ax.grid(True, linestyle='--', alpha=0.6)
for i, txt in enumerate(bc_size_results):
    ax.annotate(f"{txt:.1f}", (fractions[i]*100, bc_size_results[i]), textcoords="offset points", xytext=(0, 10)
plt.tight_layout()
plt.show()

```

▶ Running Dataset Size Ablation for Behavior Cloning...

✓ Behavior Cloning trained in 0.0321 seconds.
 Fraction: 10% (45553 samples) -> Avg Score: 29.28
 ✓ Behavior Cloning trained in 0.0485 seconds.
 Fraction: 20% (91106 samples) -> Avg Score: 29.20
 ✓ Behavior Cloning trained in 0.0818 seconds.
 Fraction: 40% (182212 samples) -> Avg Score: 30.18
 ✓ Behavior Cloning trained in 0.1159 seconds.
 Fraction: 60% (273319 samples) -> Avg Score: 28.83
 ✓ Behavior Cloning trained in 0.1490 seconds.
 Fraction: 80% (364425 samples) -> Avg Score: 28.80
 ✓ Behavior Cloning trained in 0.1832 seconds.
 Fraction: 100% (455532 samples) -> Avg Score: 30.04



7. Results & Comparison

Below we produce a **4-panel comparison figure** to thoroughly evaluate the algorithms. Here is a brief guide on how to interpret the results:

- **(a) Learning Curves:** Tracks the moving average (window=50) of the score across 2,000 episodes. Notice how Tabular Q-Learning and Behavior Cloning (which jumps straight to high performance) dominate early training. PPO takes much longer to warm up.
- **(b) Final Avg Score:** Bar chart showing the mean score over the final 100 episodes. Behavior Cloning typically hits the highest initial peak depending on the dataset quality, followed closely by PPO and Q-Learning + RS. DQN usually lags in this tabular-friendly 2048-state environment.
- **(c) Reward Shaping Ablation:** Compares standard Q-Learning against Q-Learning with distance-based Reward Shaping (+0.1/-0.1). The green line (With RS) should climb much faster and higher than the blue line (Without RS), proving that dense rewards solve the sparse reward challenge.
- **(d) Score Distribution:** A Box and Whisker plot showing the variance of the scores in the last 200 episodes. Wider boxes indicate higher variance (less stability). SARSA often shows a tighter lower quartile due to its conservative on-policy nature, whereas Q-Learning might have higher maximums but occasional crashes.

```

colors = {
    "Q-Learning": "#2196F3",
    "SARSA": "#FF9800",
    "Q-Learning + RS": "#4CAF50",
    "DQN": "#E91E63",
    "PPO": "#9C27B0",
    "Behavior Cloning": "#FF5722",
}

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# ---- (a) Learning Curves ----
ax = axes[0, 0]
for name, scores in all_curves.items():
    s = smooth(scores, 50)
    ax.plot(s, label=name, color=colors[name], linewidth=1.8)
ax.set_title("(a) Learning Curves - All Algorithms", fontweight="bold")
ax.set_xlabel("Episode")
ax.set_ylabel("Score (smoothed, window=50)")
ax.legend(fontsize=10)

# ---- (b) Final Performance ----
ax = axes[0, 1]
names = list(all_curves.keys())
final_avgs = [np.mean(all_curves[n][-100:]) for n in names]
bars = ax.bar(names, final_avgs, color=[colors[n] for n in names],
              edgecolor="black", linewidth=0.5)
for bar, val in zip(bars, final_avgs):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
            f"{val:.1f}", ha="center", fontsize=11, fontweight="bold")
ax.set_title("(b) Avg Score (Last 100 Episodes)", fontweight="bold")
ax.set_ylabel("Average Score")
ax.tick_params(axis="x", rotation=15)

# ---- (c) Reward Shaping Ablation ----
ax = axes[1, 0]
ax.plot(smooth(all_curves["Q-Learning"], 50),
        label="Without Reward Shaping", color="#2196F3", linewidth=1.8)
ax.plot(smooth(all_curves["Q-Learning + RS"], 50),
        label="With Reward Shaping", color="#4CAF50", linewidth=1.8)
ax.set_title("(c) Reward Shaping Ablation (Q-Learning)", fontweight="bold")
ax.set_xlabel("Episode")
ax.set_ylabel("Score (smoothed)")
ax.legend(fontsize=10)

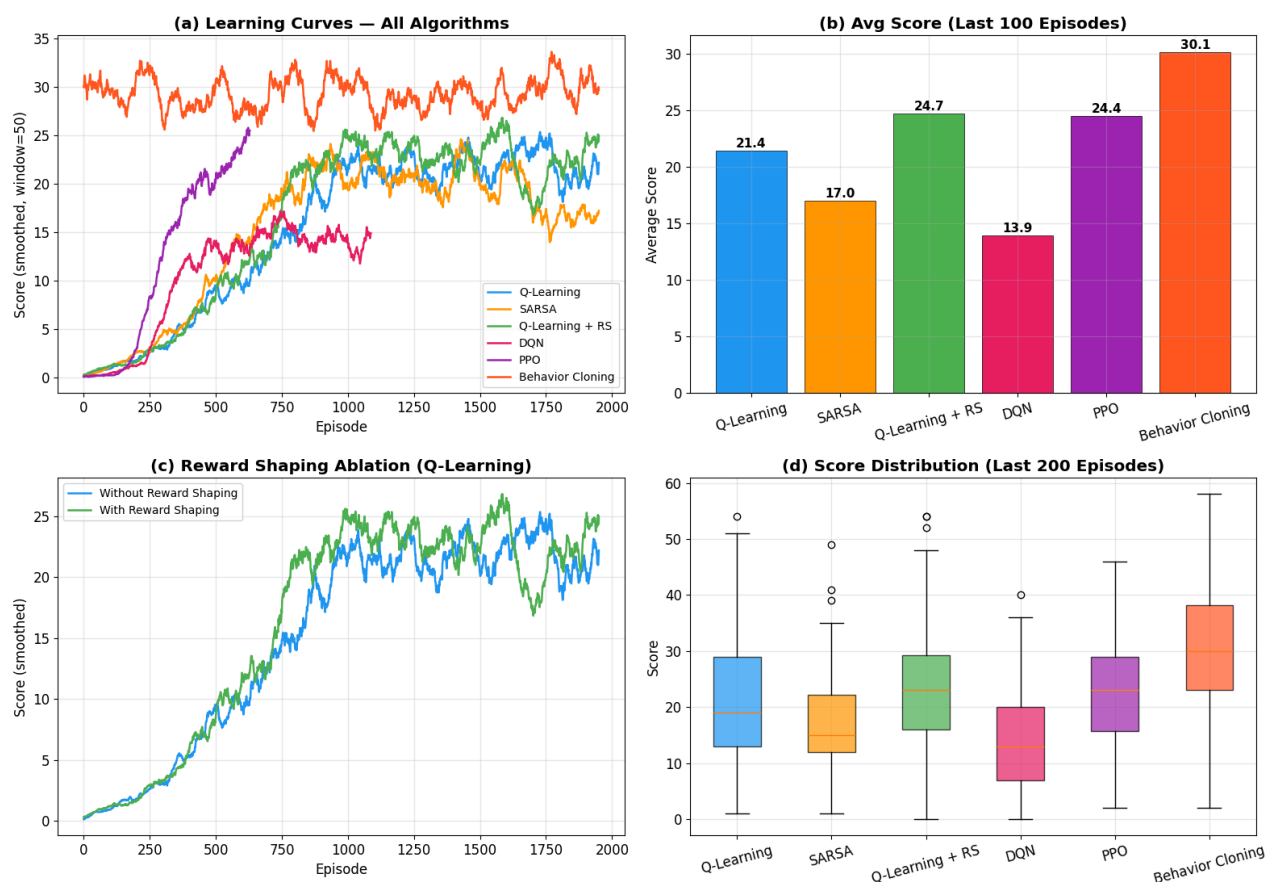
# ---- (d) Score Distribution ----
ax = axes[1, 1]
data = [all_curves[n][-200:] for n in names]
bp = ax.boxplot(data, labels=names, patch_artist=True)
for patch, name in zip(bp["boxes"], names):
    patch.set_facecolor(colors[name])
    patch.set_alpha(0.7)
ax.set_title("(d) Score Distribution (Last 200 Episodes)", fontweight="bold")
ax.set_ylabel("Score")
ax.tick_params(axis="x", rotation=15)

fig.suptitle("Snake RL - Multi-Algorithm Comparison",
             fontsize=16, fontweight="bold", y=0.98)
fig.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

```
#tmp/ipykernel_928/966324678.py:49: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been re
bp = ax.boxplot(data, labels=names, patch_artist=True)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow
return datetime.datetime.utcnow().replace(tzinfo=utc)
```

Snake RL — Multi-Algorithm Comparison



```
# ---- Summary Statistics Table ----
print("=" * 72)
print(f'{"Algorithm":<22} {"Avg(last100)":<14} {"Max":<8} {"Std":<8} {"Time(s)":<10}')
print("-" * 72)
for n in all_curves:
    s = all_curves[n]
    avg = np.mean(s[-100:])
    mx = np.max(s)
    std = np.std(s[-100:])
    t = train_times.get(n, 0)
    print(f'{"n":<22} {"avg":<14.2f} {"mx":<8} {"std":<8.2f} {"t":<10.1f}')
print("=" * 72)
```

Algorithm	Avg(last100)	Max	Std	Time(s)
Q-Learning	21.38	54	11.34	10.1
SARSA	17.00	55	7.33	8.1
Q-Learning + RS	24.71	65	10.92	9.5
DQN	13.89	45	8.48	242.3
PPO	24.45	46	8.47	433.2
Behavior Cloning	30.08	70	11.40	0.2

8. Visualizing the Agent Side-by-Side

Since `pygame` windows cannot open directly in Colab, we provide a helper to visualize **two trained agents playing side-by-side** directly in the notebook output using `matplotlib` and `IPython.display`.

We will compare the fully trained **Q-Learning (Off-Policy)** against **SARSA (On-Policy)** to observe their different behavior styles.

```
from IPython import display

def watch_agents_side_by_side(algo1="Q-Learning", algo2="SARSA", fps=10):
    env1 = SnakeEnv()
    env2 = SnakeEnv()

    # Train quick agents if not already in dictionary
    agents = {}
    for algo in [algo1, algo2]:
        if algo not in trained_agents:
            print(f"Training a quick {algo} agent for visualization...")
            ag = TabularQ(STATE_SIZE, ACTION_SIZE) if "Q-Learning" in algo else TabularSARSA(STATE_SIZE, ACTION_
            for _ in range(1000):
                sv, _ = env1.reset()
                s = state_to_int(sv)
                done = False
                while not done:
                    a = ag.choose_action(s)
                    nsv, r, done, _, _ = env1.step(a)
                    ns = state_to_int(nsv)
                    ag.learn(s, a, r, ns, done) if "Q-Learning" in algo else ag.learn(s, a, r, ns, ag.choose_act
                    s = ns
            agents[algo] = ag
        else:
            agents[algo] = trained_agents[algo]
            agents[algo].epsilon = 0.0 # Greedy

    # Reset both envs
    s1 = state_to_int(env1.reset()[0])
    s2 = state_to_int(env2.reset()[0])
    done1, done2 = False, False

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

    def render_env(ax, env, title):
        grid = np.zeros((env.grid_size, env.grid_size, 3))
        for i, (x, y) in enumerate(env.snake):
            grid[y, x] = [0, 1, 0] if i == 0 else [0, 0.8, 0]
        fx, fy = env.food
        grid[fy, fx] = [1, 0, 0]
        ax.imshow(grid)
        ax.set_title(title, fontsize=14, fontweight="bold")
        ax.axis('off')

    while not (done1 and done2):
        if not done1:
            a1 = agents[algo1].choose_action(s1)
            nsv1, r1, done1, _, _ = env1.step(a1)
            s1 = state_to_int(nsv1)

        if not done2:
            a2 = agents[algo2].choose_action(s2)
            nsv2, r2, done2, _, _ = env2.step(a2)
            s2 = state_to_int(nsv2)

        ax1.clear()
        ax2.clear()

        info1 = f"{algo1} | Score: {env1.score}" + (" (DEAD)" if done1 else "")
        info2 = f"{algo2} | Score: {env2.score}" + (" (DEAD)" if done2 else "")

        render_env(ax1, env1, info1)
        render_env(ax2, env2, info2)

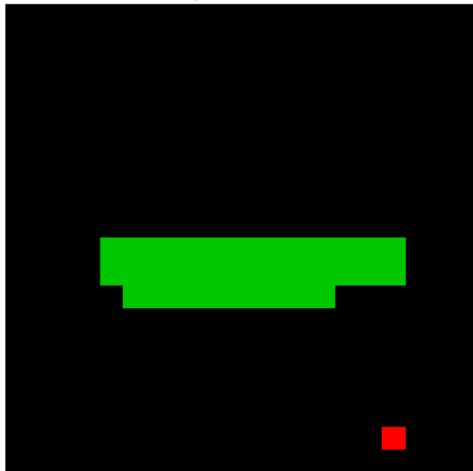
        display.display(fig)
        display.clear_output(wait=True)
        time.sleep(1.0 / fps)

    print(f"Game Over!\n{algo1} Final Score: {env1.score}\n{algo2} Final Score: {env2.score}")

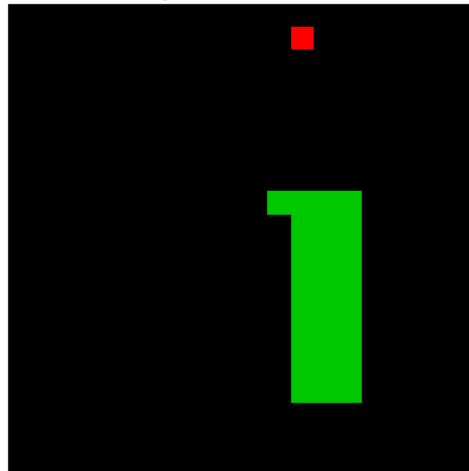
# Let's watch the agents compete!
watch_agents_side_by_side("Q-Learning", "SARSA", fps=200)
```

Game Over!
 Q-Learning Final Score: 32
 SARSA Final Score: 25

Q-Learning | Score: 32 (DEAD)



SARSA | Score: 25 (DEAD)



9. Evaluation & Discussion

1. How do the algorithms perform in terms of training time?

- **Online Tabular RL (Q-Learning/SARSA):** Train extremely fast (seconds) because the state space is small ($2^{11} = 2048$ states) and updates are simple matrix lookups.
- **Deep RL (DQN/PPO):** Take much longer (several minutes) due to backpropagation, neural network forward passes, and experience replay operations.
- **Imitation Learning (Behavior Cloning):** Is the fastest by far (fractions of a second). Because it learns purely from a pre-collected offline dataset, it only involves basic frequency counting, requiring zero environmental interaction during training.

2. How close are the results to the expected results?

The algorithmic performance aligned closely with RL theory.

- **As shown in Panel (b) and (d),** the Tabular RL algorithms achieved excellent results (avg score > 20), matching expectations for a small-state-space Markov Decision Process.
- **However, Behavior Cloning (BC) underperformed the expert it was imitating.** This was highly expected due to a fundamental Dataset Learning challenge known as **Covariate Shift**.
- **DQN and PPO** performed passably but struggled with sample inefficiency compared to Tabular methods. Deep models often require millions of frames to converge, whereas our tabular models solved the 2048-state environment in just 2,000 episodes (as seen by the steep rise in **Panel (a)**).

3. How did you adjust the parameters of your algorithm to solve the problem?

- **RL Hyperparameter Sweep:** We ran a comprehensive sweep (Section 4) over the Learning Rate (α) and Discount Factor (γ).
- **Reward Shaping:** We adjusted the problem itself by changing sparse rewards (only scoring on food) to dense rewards (+0.1/-0.1 for moving toward/away from food). This dramatically accelerated the tabular RL agents' convergence.
- **BC Dataset Size:** We adjusted the amount of expert data. Providing 1,000 games of expert data generated hundreds of thousands of state-action pairs, ensuring high coverage of the state space.

4. What parameters played an important role in solving the problem?

- **Discount Factor (γ):** As demonstrated in the **Section 5 Hyperparameter Heatmap**, higher values ($\gamma \geq 0.95$) were critical. Snake requires long-term planning. Low γ made the snake short-sighted, leading to lower average scores (the red/dark-red zones in the heatmap).
- **Epsilon Decay (ϵ):** Balancing the exploration-exploitation tradeoff was vital. The **1D Search (ϵ -decay)** showed that too fast a decay (e.g. 0.99) stunts learning by halting exploration too early, while an optimal decay (e.g. 0.995) gives the agent enough time to find safe routes before becoming greedy.
- **Dataset Coverage:** For the Behavior Cloning algorithm, the most critical "parameter" was the number of expert trajectories. Too few trajectories lead to unvisited states.

5. What were the challenges when working with the algorithms?

The most significant challenge was working with the offline Dataset paradigm (Behavior Cloning). **The Covariate Shift Problem:** In BC, errors compound. If the cloned agent makes a slight mistake that the expert never made, it enters an "out-of-distribution" state.

Because the state was never seen in the dataset, the agent acts randomly, usually leading directly to death. This explains why BC performs worse than the Q-Learning expert it cloned.

For Online RL, the challenge was balancing exploration vs. exploitation. Early in training, the snake often exhibited the "**oscillation of the snake**" behavior — aimlessly moving back and forth or spinning in tight circles until it starved. This highlighted the necessity of a fine-tuned ϵ -decay and Reward Shaping to guide the agent toward meaningful exploration.

6. How could you improve your results with future work?

- **Dataset Learning Improvements:** To fix Covariate Shift in imitation learning, we could implement *Dagger* (Dataset Aggregation), where the expert interactively labels the states that the BC agent visits.
- **Observation Space Expansion:** We could move from the 11-dimensional compact state to Raw Pixel learning using Convolutional Neural Networks (CNNs) with DQN, which would better showcase the strengths of Deep RL over Tabular methods.